PasswordStore Vulnerability Assessment

Version 1.0

Prepared by: Cyfrin.io-Student-Elechukwu

Date: January 22, 2026

Prepared By

Auditor: Cyfrin.io-Student-Elechukwu
Affiliation: Cyfrin

- https://cyfrin.io
- https://github.com/Azel-C/Blockchain-Training

Lead Auditors:

- xxxxxxx

Table of Contents

Protocol Summary

PasswordStore is a protocol dedicated to the storage and retrieval of user passwords.
The protocol is designed to be used by a **single user**, not multiple users.

Only the owner should be able to:

- Set the password
- Read the password

Disclaimer

The auditor has made reasonable efforts to identify potential security vulnerabilities within the scope and time constraints of this engagement. However, no guarantee is made that all vulnerabilities have been identified.

A security audit is **not** an endorsement of the underlying business, product, or its future performance.
The review was time-boxed and focused solely on the **security aspects** of the Solidity smart contract implementation based on the provided codebase.

Risk Classification

Likelihood \ Impact	High	Medium	Low
High	H	H / M	M
Medium	H / M	M	M / L
Low	M	M / L	L

Severity ratings follow the **CodeHawks Severity Matrix**:
https://docs.codehawks.com/hawks-auditors/how-to-evaluate-a-finding-severity

---

Audit Details

Audit Commit Hash:

```
2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Scope

```
./src/
└── PasswordStore.sol
```

Roles

- **Owner:** Can set and read the password
- **Outsider:** Should not be able to read or modify the password

---

Executive Summary

Issues Found

Severity	Number of Issues
High	2
Medium	0
Low	0
Informational	1
Total	**3**

Findings

High Severity

H-1: Storing the password on-chain makes it visible to anyone

Severity: High
Impact: High
Likelihood: High

Description

All data stored on-chain is publicly readable.
The `PasswordStore::s_password` variable is intended to be private and accessed only via `getPassword`, but blockchain storage can be read directly by anyone.

Impact

Any attacker can read the private password, severely breaking the protocol's intended functionality.

Proof of Concept

The following steps demonstrate how the password can be read directly from storage.

1. Start a local chain:

```
make anvil
```

2. Deploy the contract:

```
make deploy
```

3. Read storage slot `1`:

```
cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

Example output:

```
0x6d7950617373776f726400000000000000000000000000000000000000000014
```

4. Decode the value:

```
cast parse-bytes32-string
0x6d7950617373776f726400000000000000000000000000000000000000000014
```

Decoded output:

```
myPassword
```

Recommended Mitigation

The overall architecture should be reconsidered.

- Encrypt the password **off-chain**
- Store only the encrypted value on-chain
- Remove public view accessors to prevent accidental disclosure

---

H-2: Missing access control on `setPassword`

Severity: High **Impact:** High **Likelihood:** High

Description

The `setPassword` function is declared `external` and lacks access control, despite its NatSpec stating that only the owner should be able to call it.

```
function setPassword(string memory newPassword) external {
    // @audit: No access control
    s_password = newPassword;
    emit SetNetPassword(); }
```

Impact

Any address can overwrite the stored password, fully compromising the contract.

Proof of Concept

Add the following test to `PasswordStore.t.sol`:

▶ Code

```
function test_anyone_can_set_password(address randomAddress) public {
        vm.assume(randomAddress != owner);
        vm.prank(randomAddress);
        string memory newPassword = "hackedPassword";
        passwordStore.setPassword(newPassword);

        vm.prank(owner);
        string memory actualPassword = passwordStore.getPassword();
        assertEq(actualPassword, newPassword);
    }
```

Recommended Mitigation

Restrict access to the owner:

```
if (msg.sender != s_owner) {
    revert PasswordStore_NotOwner();
}
```

Informational

I-1: Incorrect NatSpec documentation

Severity: Informational

Description

The NatSpec for `getPassword` references a parameter that does not exist.

```
/*
 * @notice This allows only the owner to retrieve the password.
 * @param newPassword The new password to set.
 */
function getPassword() external view returns (string memory) {
```

Impact

The documentation is misleading and incorrect.

Recommended Mitigation

Remove the incorrect NatSpec line:

```
- * @param newPassword The new password to set.
```

End of Report